

18

Writing Integration Tests Using xUnit

In the previous chapter, we wrote input field validations in our web app's forms to improve the **user experience (UX)** when filling out forms. Now, we are going to add tests to our web service.

Writing automated tests is a big topic—too big to be written in a single book and to discuss all the details you need to know. However, we are only going to scratch the surface and write practical tests. We will write a few unit tests and then write integration tests in our ASP.NET Core application.

We will cover the following topics in this chapter:

- Getting started with automated testing
- Installing **Microsoft SQL Server (MS SQL Server)** in a Docker container
- Understanding xUnit
- Understanding unit testing
- Understanding integration testing

Technical requirements

Here is what you need to complete this chapter:

- Visual Studio 2019
- Visual Studio for Mac
- JetBrains Rider
- A Docker client
- The **Entity Framework Core (EF Core) command-line interface (CLI)**

Here's a link to the finished repository of this chapter: <https://github.com/PacktPublishing/ASP.NET-Core-and-Vue.js/tree/master/Chapter18/>

Getting started with automated testing

Let's first define what automated testing is. **Automated testing** is a process of writing code to test the code in our project and then running those tests in an automated way—that's it.

Imagine this scenario: if you want to test a function manually, you have to launch your application in the browser, fill in the form to authenticate, and click some links to get to the function you want to test. You will then have to fill out all the fields in your form, hit the **Submit** button, and see the function's feedback or results in the browser. Imagine you have to repeat how you've manually tested your form and also try to test your form in different edge cases. That's crazy! Simply because of the variations in how your user would use your form, this grows exponentially as you add more fields to your form.

As you can see, manual testing is very time-consuming. The workflow for manually testing your function may take several minutes every time and would be much worse if you had hundreds of functions in your application—imagine that.

So, what will you get out of automated testing?

Benefits of automated testing

Here are the reasons why you should write tests in your application:

- **Test your code frequently, in less time:** You can test your application code often and in less time.
- **Catch bugs before deploying:** Catch bugs before deploying your application.

- **Deploy with confidence:** This is extremely important because it allows you to deploy your application with more confidence.
- **Code that works after refactoring:** The problem with refactoring code that has not been tested is that you have to manually test the code just to check that it is working again.

This process is excruciating if you are going to refactor hundreds of methods or functions because a) it's time-consuming, and b) as your application grows, you may forget about the parts that need to be tested. With automated tests, every time you refactor your code, you can easily run your tests to see if you broke something.

- **Focus more on the quality:** It helps if you focus more on the quality of the methods or functions you are writing, making sure that every method works with different inputs in varying circumstances.

Here's a fun fact: the mentioned benefits of automated testing help me sleep at night.

Going back to our topic, to set up our automated testing we will replace SQLite with MS SQL Server to write integration tests later. There are limitations to SQLite, which we are currently using in our ASP.NET Core application—limitations in terms of modeling, querying, and migrations. We have used SQLite in the previous chapters to avoid spending time setting up a database with a real database engine in Windows, Linux, or Mac OS X and to instead focus on the architecture of our application.

To start with, Windows users can choose between installing an MS SQL Server instance on their Windows machines or in Docker containers. In comparison, Linux and Mac OS X users will only have to use Docker to install a MS SQL Server instance.

The following screenshot shows **SQL Server Management Studio (SSMS)** for Windows machines:

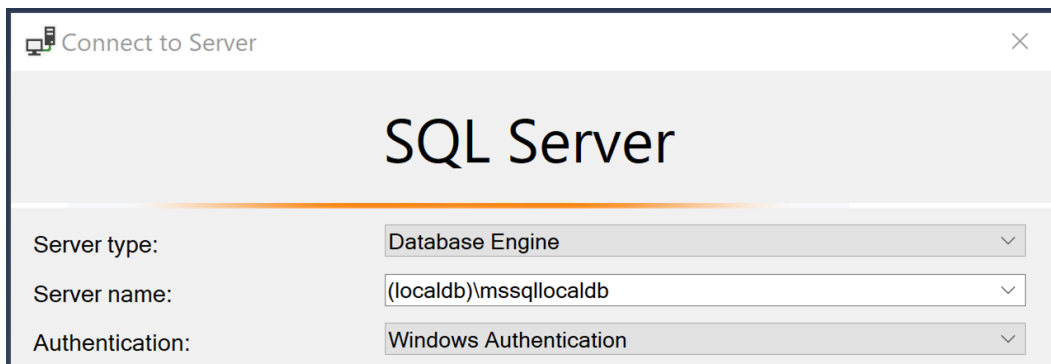


Figure 18.1 – SSMS

With that, here are two different connection strings, one for Windows and another one for Docker.

For MS SQL Server in Windows, here is the connection string:

```
"DefaultConnectionUsingWindows": "Server=(localdb)\\  
mssqllocaldb;Database=TravelDb;Trusted_  
Connection=True;MultipleActiveResultSets=true"
```

Replace the current value of the database connection string in `appsettings.json` with the preceding value.

For MS SQL Server in Docker, here is the connection string:

```
"DefaultConnectionUsingDocker": "Data  
Source=localhost,1433;Initial Catalog=TravelDb;User  
id=sa;Password=Pass123!;MultipleActiveResultSets=True"
```

Replace the current value of the database connection string in `appsettings.json` with the preceding value.

In the next section, we will install MS SQL Server in Docker.

Installing MS SQL Server in a Docker container

Before we use **Docker** and install MS SQL Server in it, let's have a quick discussion about Docker. If you haven't tried Docker before, Docker is a **platform as a service (PaaS)** that provides **operating system (OS)**-level virtualization to deliver software installation in packages called **containers**.

I would suggest reading more about Docker and containers at <https://www.docker.com/resources/what-container> because we will stay focused on automated testing topics in this chapter.

Here are the links for installing Docker:

- **Installing Docker Engine on Ubuntu:** <https://docs.docker.com/engine/install/ubuntu/>
- **Installing Docker Desktop on Mac:** <https://docs.docker.com/docker-for-mac/install/>
- **Installing Docker Desktop on Windows:** <https://docs.docker.com/docker-for-windows/install/>

After installing Docker on your Linux machine, MacBook, or Windows, you can install MS SQL Server in Docker. Here's a quick guide on how to do it: <https://devlinduldulao.pro/how-to-use-microsoft-sql-server-on-mac-for-development>.

It seems as though the guide is only for Mac users, but trust me—the installation of MS SQL Server in Docker from the guide is also applicable to Ubuntu and Windows users.

Next up is Azure Data Studio, a cross-platform database tool that is an alternative to SSMS.

The preceding link to the guide on using MS SQL Server on a Mac also provides details of how to use Azure Data Studio.

Next, after installing Docker, MS SQL Server, and Azure Data Studio, let's install the NuGet package of the EF SQL Server provider.

Install the `Microsoft.EntityFrameworkCore.SqlServer` NuGet package in the `Travel.Data` project and then update `services.AddDbContext` in the `DependencyInjection.cs` file of the `Travel.Data` project, with the following code:

```
services.AddDbContext<ApplicationDbContext>(options =>
    options.UseSqlServer(config
        .GetConnectionString("DefaultConnection")));
```

The preceding code only replaces `UseSqlite` with `UseSqlServer`.

Next, we delete the `Migrations` directory of `Travel.Data` and create a new migration for the MS SQL Server database using the following dotnet command:

```
dotnet ef migrations add InitialCreate --startup-project ../../presentation/Travel.WebApi/
```

We are creating a new migration and syncing it with the MS SQL Server database that we will use later.

Next, we update the database by running the following command:

```
dotnet ef database update --startup-project ../../presentation/Travel.WebApi/
```

After running the preceding `dotnet ef Core` command, the MS SQL Server database should now be ready.

Let's create `DbContext` for seeding our data by creating a new C# file named `ApplicationDbContextSeed.cs` in the `Contexts` directory of the `Travel.Data` project and adding the following code:

```
namespace Travel.Data.Contexts
{
    public static class ApplicationDbContextSeed
    {
        public static async Task
            SeedSampleDataAsync(ApplicationDbContext context)
        {
            if (!context.TourLists.Any())
            {
                await context.TourLists.AddAsync(new TourList
                {
                    City = "Oslo", About = "...",
                    Country = "Norway",
                    // Removed for brevity. See Github repo
                });
                await context.SaveChangesAsync();
            }
        }
    }
}
```

You can get the sample `TourList` object's whole details with a `TourPackage` object from the finished repository of this chapter.

Now, let's create a database seeder in our `Program.cs` file by updating the code like so:

```
public static async Task<int> Main(string[] args)
{
    var name = Assembly.GetExecutingAssembly().GetName();
    Log.Logger = new LoggerConfiguration()
        .MinimumLevel.Debug()
        // Removed for brevity. See Github repo
        .WriteTo.Console().CreateLogger();
}
```

```
// creating a host builder here.
}
```

The preceding code was for the configuration of our logger. We will write our database seeder with the following code:

```
try
{
    Log.Information("Starting host");
    var host = CreateHostBuilder(args).Build();
    using (var scope = host.Services.CreateScope())
    {
        var services = scope.ServiceProvider;
        try
        {
            // Removed for brevity. See Github repo
        }
        await host.RunAsync();
        return 0;
    }
    catch (Exception ex)
    {
        Log.Fatal(ex, "Host terminated unexpectedly");
        return 1;
    }
    finally
    {
        Log.CloseAndFlush();
    }
}
```

`await ApplicationDbContextSeed.SeedSampleDataAsync(context)` will seed the sample data in the MS SQL Server database once the application is started. Build and run the application to see if the application will still run without any problems.

We are now ready to move on to the next section, which is about xUnit and writing tests.

Understanding xUnit

xUnit is an open source modern testing framework for .NET written by the author of **NUnit v2**, a unit-testing framework for all .NET languages. xUnit is the first testing framework that is compatible with .NET Core.

xUnit is the default testing tool in Visual Studio for .NET Core and is currently being used by Microsoft themselves in their modern projects.

Features of xUnit

Let's check out the features of xUnit, as follows:

- **Supports multiple platforms:** You can use it for testing .NET Framework applications, Xamarin applications, .NET Core applications, and ASP.NET Core applications.
- **Supports parallel test execution:** It speeds up the execution of unit tests.
- **Supports data-driven tests:** You can write one test and then pass different data inputs to see different expected outputs.
- **It is designed to be extensible:** You can add more data types, attributes, and asserts, and also use xUnit alongside other testing frameworks.
- **Easy to install:** You can install it via NuGet packages.

Okay—after learning the basics of xUnit, let's start using it.

Using xUnit in ASP.NET Core

Our first task is to create a folder in the root directory and name it `tests`. Go to the newly created `tests` directory and run the following command:

```
dotnet new xunit --name Application.IntegrationTests
```

The preceding command creates an `xunit` project with the name `Application.IntegrationTests` in the `tests` directory.

The command automatically adds the following packages in the `xunit` project:

- `Microsoft.NET.Test.Sdk`: This package has MSBuild targets as well as properties for building .NET test projects.
- `xunit`: This package consists of xUnit libraries for writing tests such as `xunit.core`, `xunit.assert`, and `xunit.analyzers` tests.

- `xunit.runner.visualstudio`: This package is the **Test Explorer** runner for the xUnit testing framework.

Now, go to the `Application.IntegrationTests` project folder, like so:

```
cd Application.IntegrationTests
```

Then, add a reference to the `Travel.WebApi` project, as follows:

```
dotnet add reference ../../src/presentation/Travel.WebApi/  
Travel.WebApi.csproj
```

Now, let's add the following NuGet packages to `Application.IntegrationTests`:

- `FluentAssertions`: An elegant and better way of writing assertions in our tests
- `Moq`: The most friendly and popular mocking library for .NET
- `Respawn`: Resets test databases to a clean state

After installing the NuGet packages, go to the `tests` directory again and run the following `dotnet` command:

```
dotnet new xunit --name Application.UnitTests
```

The preceding command creates a new xUnit project called `Application.UnitTests` in the `tests` directory.

Then, go to `Application.UnitTests` by running the following command:

```
cd Application.UnitTests
```

Next, add a reference to the `Travel.Application` project, as follows:

```
dotnet add reference ../../src/core/Travel.Application/Travel.  
Application.csproj
```

Now, let's add a NuGet package to `Application.UnitTests`. We are going to add a package called `FluentAssertions`.

Now, go back to the `Travel` root directory where the solution file is located and run the following `dotnet` command:

```
dotnet sln add tests/Application.IntegrationTests/Application.  
IntegrationTests.csproj
```

The preceding command adds the `Application.IntegrationTest` project to the solution file of our ASP.NET Core application.

Let's also add the `Application.UnitTests` project to our solution, as follows:

```
dotnet sln add tests/Application.UnitTests/Application.
UnitTests.csproj
```

We are only going to have a few unit tests for `Travel.Application`—the rest are integration tests for `Travel.Application` as well.

The folder structure of the application should look like this:

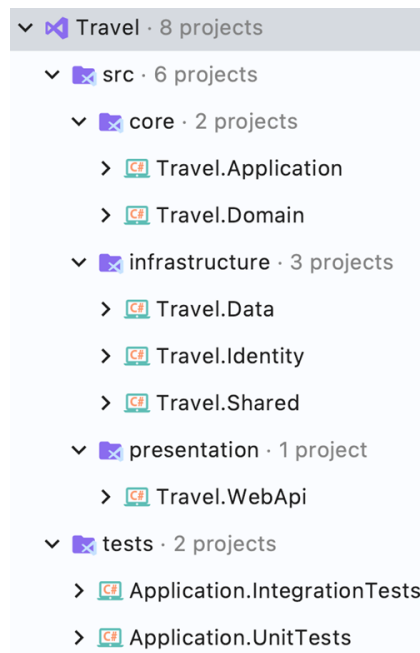


Figure 18.2 – Folder structure after adding tests projects

The preceding screenshot shows the projects in our **integrated development environment (IDE)**, where `Application.IntegrationTests` and `Application.UnitTests` are inside the `tests` directory.

Now, we are ready to go to the unit testing and integration testing sections. However, before proceeding, I highly recommend checking out the `FluentAssertions` website (<https://fluentassertions.com/>) and reading the documentation so that you have an idea of what `FluentAssertions` can do, as we will be using this to replace the built-in `Assert` **application programming interface (API)** of `xUnit`.

Understanding unit testing

Unit tests test a unit of an application without external dependencies such as message queues, files, databases, web services, and so on. You can run hundreds of unit tests in just a few seconds because they are cheap to write and quick to execute, hence you can easily verify that each building block in your application is working as expected. Consequently, since you are not testing your class or components with their external dependencies, you can't gain a lot of confidence in your application's reliability. This is where integration tests come into play, which we will talk about later.

In my day-to-day work, I only write tests for complex code that is error-prone runtime code, complex code, and algorithmic logic, and then I put all my effort into integration tests. Anyway, I have other opinions about unit testing, but you will see these later. For now, let's start writing some unit tests using `xUnit` and `FluentAssertions`.

Writing unit tests

Create a new folder called `Common` in the root directory of the `Application`. `UnitTests` project. Then, create two folders named `Exceptions` and `Mappings` inside the newly created `Common` folder.

Tests for exception

Create a C# file named `ValidationExceptionTests.cs` inside the newly created `Exceptions` folder and add the following code:

```
using System;
using System.Collections.Generic;
using FluentAssertions;
using FluentValidation.Results;
using Travel.Application.Common.Exceptions;
using Xunit;

namespace Application.UnitTests.Common.Exceptions
{
    public class ValidationExceptionTests
    {
        [Fact]
        public void
            DefaultConstructorCreatesAnEmptyErrorDictionary()
        {
            var actual = new ValidationException().Errors;
```

```
        actual.Keys.Should()  
            .BeEquivalentTo(Array.Empty<string>());  
    }  
    // another test method here.  
}  
}
```

What's happening in the preceding code? The `Fact` attribute is used to write unit tests that have no method arguments. The first unit test checks the default constructor of `ValidationException` and creates an empty array dictionary. The `Validation` class contains a dictionary of errors. Then, we can assert the test by using the `FluentAssertions` library.

Here is the other method:

```
[Fact]  
public void SingleValidationFailure  
    CreatesASingleElementErrorDictionary()  
{  
    var failures = new List<ValidationFailure>  
    {  
        new ("Mobile", "Mobile is required.")  
    };  
    var actual = new ValidationException(failures)  
        .Errors;  
    actual.Keys.Should()  
        .BeEquivalentTo("Mobile");  
    actual["Mobile"].Should()  
        .BeEquivalentTo("Mobile is required.");  
}
```

`Should().BeEquivalentTo()` is the extension method from the `FluentAssertions` library that compares the object, which calls this method with the value passed into `BeEquivalentTo`. Then, the assertion will return `true` if both are equal.

The `FluentAssertions` library is preferred when comparing values other than strings. `Should().BeEquivalentTo()` is an extension method that can be used with different data types.

Although we can also use xUnit default methods such as `Assert.True()`, where we have to pass a condition that returns `true` or `false`, I prefer the `FluentAssertions` extension method over `Assert.True()` since it makes the comparison of the two states both easy and readable.

Tests for mapping

Create a C# file named `MappingTests.cs` inside the newly created `Mappings` folder and then add the following code:

```
using System;
using AutoMapper;
using Travel.Application.Common.Mappings;
using Travel.Application.Dtos.Tour;
using Travel.Domain.Entities;
using Xunit;
namespace Application.UnitTests.Common.Mappings
{
    public class MappingTests
    {
        // Removed for brevity. Go to this repo's Github
        [Fact]
        public void ShouldHaveValidConfiguration()
        {
            _configuration.AssertConfigurationIsValid();
        }
        [Theory]
        [InlineData(typeof(TourList), typeof(TourListDto))]
        [InlineData(typeof(TourPackage),
            typeof(TourPackageDto))]
        public void ShouldSupportMappingFromSourceTo
            Destination(Type source, Type destination)
        {
            var instance = Activator.CreateInstance(source);
            _mapper.Map(instance, source, destination);
        }
    }
}
```

So, what have you noticed in the preceding code? There are `Theory` and `InlineData` attributes. The `Theory` attribute is used to write unit tests whereby we pass a set of parameters to the method, while the `InlineData` attribute is used to give data to the method parameters.

We can write the `InlineData` attribute multiple times. xUnit will create multiple instances of tests and populate them with test case arguments, which we have passed in `InlineData`.

The last unit test with `InlineData` checks whether our `mapper` attribute supports the mapping of `TourList` to `TourListDto` and `TourPackage` to `TourPackageDto`.

We have a total of five unit tests in our code. The last unit test was multiplied by two because we are using two `InlineData` attributes.

Okay—so, you've now seen how to write unit tests using xUnit and `FluentAssertions`. It's not that complicated. Now, we can proceed to the next section, which is about integration testing.

Understanding integration testing

What is an integration test? **Integration tests** test an application with its external dependencies by testing your application code's integration with concrete dependencies such as files, databases, and so on.

Integration tests take longer to execute because they often involve reading or writing to a database, but in return they give us more confidence in our application's health.

If a fast feedback loop from unit tests could send misleading results, I wouldn't mind writing a slow integration test that provides a correct outcome. A tricky feedback loop isn't worth the speed.

Enough about my opinions—let's start writing some integration tests and test all the commands and queries of our MediatR implementation.

Writing integration tests

Our first task is to create a new `appsettings.json` file in the root folder of `Application.IntegrationTests`. Copy the content of the `Travel.WebApi's` `appsettings.json` file and paste it into the newly created `appsettings.json` file, but change the database name from `TravelDb` to `TravelTestDb` in the connection strings.

Next, we will create a test fixture so that we can share a resource for several tests.

Creating a test fixture

Create a C# file named `DatabaseFixture.cs` in the root folder of `Application.IntegrationTests` and add the following libraries and packages:

```
using System;
using System.IO;
using System.Threading.Tasks;
using MediatR;
using Microsoft.AspNetCore.Hosting;
using Microsoft.EntityFrameworkCore;
using Microsoft.Extensions.Configuration;
using Microsoft.Extensions.DependencyInjection;
using Moq;
using Respawn;
using Travel.Data.Contexts;
using Travel.WebApi;
```

Now, let's define a `DatabaseFixture` class that implements `IDisposable`, like so:

```
namespace Application.IntegrationTests
{
    public class DatabaseFixture : IDisposable
    {
        private static IConfigurationRoot _configuration;
        private static IServiceScopeFactory _scopeFactory;
        private static Checkpoint _checkpoint;
        public DatabaseFixture()
        {
            var builder = new ConfigurationBuilder()
                .SetBasePath(Directory.GetCurrentDirectory())
                .AddJsonFile("appsettings.json", true, true)
                .AddEnvironmentVariables();
            _configuration = builder.Build();
            var startup = new Startup(_configuration);
            var services = new ServiceCollection();
```

```

        services.AddSingleton(
            Mock.Of<IWebHostEnvironment>(w =>
                w.EnvironmentName == "Development" &&
                w.ApplicationName == "Travel.WebApi"));
        services.AddLogging();
        startup.ConfigureServices(services);
        _scopeFactory = services
            .BuildServiceProvider()
            .GetService<IServiceScopeFactory>();
        _checkpoint = new Checkpoint
        {
            TablesToIgnore = new[] {
                "__EFMigrationsHistory"
            };
        };
        EnsureDatabase();
    } // static methods and IDispose are below ..
}

```

So, what did we write in the preceding code?

We created a configuration called `ConfigurationBuilder`, and services named `ServiceCollection`. We created a web hosting environment using `Moq` to run the application in the `Travel.WebApi` project. We also added a checkpoint for resetting the database state.

Adding static methods

Add the static methods, as shown in the following code snippet:

```

private static void EnsureDatabase()
{
    using var scope = _scopeFactory.CreateScope();
    var context = scope.ServiceProvider
        .GetService<ApplicationDbContext>();
    context.Database.Migrate();
}

public static async Task ResetState()
{

```

```

        await _checkpoint.Reset(_configuration
            .GetConnectionString("DefaultConnection"));
    }

    public static async Task<TResponse>
    SendAsync<TResponse>(IRequest<TResponse> request)
    {
        using var scope = _scopeFactory.CreateScope();
        var mediator = scope.ServiceProvider
            .GetService<IMediator>();
        return await mediator.Send(request);
    }

```

We wrote a static method called `EnsureDatabase` that runs migrations for our database, and another static method named `ResetState` for resetting the database state before each test. Multiple database operations will not affect each other's state.

Here are the last two static methods. You know the drill, so add them as well. The code is shown in the following snippet:

```

    public static async Task AddAsync<TEntity>(TEntity entity)
    where TEntity : class
    {
        using var scope = _scopeFactory.CreateScope();
        var context = scope.ServiceProvider
            .GetService<ApplicationDbContext>();
        context.Add(entity);
        await context.SaveChangesAsync();
    }

    public static async Task<TEntity> FindAsync<TEntity>(int id)
    where TEntity : class
    {
        using var scope = _scopeFactory.CreateScope();
        var context = scope.ServiceProvider
            .GetService<ApplicationDbContext>();
        return await context.FindAsync<TEntity>(id);
    }

    public void Dispose()
    {

```

```
// Code to run after all tests  
}
```

The last two static methods are for adding and finding an entity. Lastly, the `Dispose` method is a global teardown that gets called after every test method.

Using the test fixture

To use the `DatabaseFixture` class that we created, let's create a C# file named `DatabaseCollection.cs` in the root folder of `Application.IntegrationTests` again and add the following code:

```
using Xunit;  
namespace Application.IntegrationTests  
{  
    [CollectionDefinition("DatabaseCollection")]  
    public class DatabaseCollection :  
        ICollectionFixture<DatabaseFixture>  
    {  
    }  
}
```

The class has a `CollectionDefinition` attribute and implements an `ICollectionFixture` type called `DatabaseFixture`.

You will notice there is no code in the `DatabaseCollection` class because it is only used to define the collection. Moreover, the string in the `CollectionDefinition` attribute gives a unique name for the test collection identification.

Next, create a `TourLists` folder in the root directory of the `Application.IntegrationTests` project. Then, create two folders called `Commands` and `Queries` inside the newly created `TourLists` folder.

TourLists/Queries/GetTourTests

Now, let's create a C# file named `GetToursTests.cs` inside the `Queries` folder and add the following code:

```
using System.Collections.Generic;  
using System.Threading.Tasks;  
using FluentAssertions;  
using Travel.Application.TourLists.Queries.GetTours;
```

```

using Travel.Domain.Entities;
using Xunit;
namespace Application.IntegrationTests.TourLists.Queries
{
    using static DataFixture;
    [Collection("DatabaseCollection")]
    public class GetToursTests
    {
        public GetToursTests()
        {
            ResetState().GetAwaiter().GetResult();
        }

        // test methods here
    }
}

```

So, what's happening in the preceding code? Firstly, we are using `static DataFixture` to access all the static methods of our base class.

Then, we have a `Collection` attribute to identify the collection this test class belongs to. We are passing a `DatabaseCollection` string to make it part of that collection.

The `GetToursTests` class will be able to share the test context of the `DataFixture` class with other tests.

And in the `GetToursTests` constructor, we call the `ResetState` method to reset the database to its original state before each test runs.

Here are the test methods for the `GetToursTests` test class. Write the following code:

```

[Fact]
    public async Task ShouldReturnTourLists()
    {
        var query = new GetToursQuery();
        var result = await SendAsync(query);
        result.Lists.Should().NotBeEmpty();
    }

[Fact]
    public async Task

```

```

        ShouldReturnAllTourListsAndPackages()
        {
            await AddAsync(new TourList
            {
                City = "Manila", Country = "Philippines",
                About = "Lorem Ipsum",
                TourPackages = new List<TourPackage>
                {
                    new()
                    {
                        Name = "Free Walking Tour Manila",
                        // Removed for brevity. Go to this repo's Github.
                    }
                }
            });

            var query = new GetToursQuery();
            var result = await SendAsync(query);
            result.Lists.Should().HaveCount(1);
        }

```

Lastly, the `ShouldReturnTourLists` test method should not be empty, while the `ShouldReturnAllTourListsAndPackages` method should have one object.

TourLists/Commands/CreateTourListTests

Next, we create a C# file named `CreateTourListTests.cs` in the `Commands` folder of the `TourLists` directory and add the following code:

```

// Removed for brevity. Go to this repo's Github.
using Travel.Application.Common.Exceptions;
namespace Application.IntegrationTests.TourLists.Commands
{
    using static DataFixture;
    [Collection("DatabaseCollection")]
    public class CreateTourListTests
    {
        public CreateTourListTests()
        {

```

```

        ResetState().GetAwaiter().GetResult();
    }
    [Fact]
    public void ShouldRequireMinimumFields()
    {
        var command = new CreateTourListCommand();
        FluentActions.Invoking(() =>
            SendAsync(command)).Should()
            .Throw<ValidationException>();
    }
    // two test methods here
}

```

As you can see in the preceding code, we reuse static `DataFixture`, the `Collection` attribute, and the `ResetState` function. Expect these through the last test class that we will create later. Anyhow, the `ShouldRequireMinimumFields` test method should throw a `ValidationException` error, the custom validation exception we've created in the `Travel.Application` project, because the test tried to create a `TourList` object without the required properties.

Here are two more test methods:

```

[Fact]
public void ShouldRequireAbout()
{
    var command = new CreateTourListCommand
    {
        City = "Antananarivo",
        Country = "Madagascar", About = ""
    };
    FluentActions.Invoking(() =>
        SendAsync(command)).Should()
        .Throw<ValidationException>();
}
[Fact]
public async Task ShouldCreateTourList()
{

```

```
var command = new CreateTourListCommand
{
    City = "Antananarivo", Country = "Madagascar",
    About = "Lorem Ipsum"
};
var id = await SendAsync(command);
var list = await FindAsync<TourList>(id);
list.Should().NotNull();
list.City.Should().Be(command.City);
list.Country.Should().Be(command.Country);
list.About.Should().Be(command.About);
}
```

The `ShouldRequireAbout` test should also throw a validation error because the `About` property is empty. And lastly, the `ShouldCreateTourList` test should create a `TourList` object because all the required properties were included.

TourLists/Commands/DeleteTourListTests

Next, we create a C# file named `DeleteTourListTests.cs` in the `Commands` folder of the `TourLists` directory and add the following code:

```
using FluentAssertions;
// Removed for brevity. Go to this repo's Github.
using Xunit;
namespace Application.IntegrationTests.TourLists.Commands
{
    using static DataFixture;
    [Collection("DatabaseCollection")]
    public class DeleteTourListTests
    {
        public DeleteTourListTests()
        {
            ResetState().GetAwaiter().GetResult();
        }
        [Fact]
        public void ShouldRequireValidTourListId()
        {

```

```

        var command = new DeleteTourListCommand
        {
            Id = 33
        };
        FluentActions.Invoking(() =>
            SendAsync(command)).Should()
            .Throw<NotFoundException>();
    }
    // another test method
}

```

ShouldRequireValidTourListId should throw a *not found* exception in the preceding code because the ID 33 does not exist, but the test method tried to delete it.

And here is the other test method:

```

[Fact]
public async Task ShouldDeleteTourList()
{
    var listId = await SendAsync(new
        CreateTourListCommand
        {
            City = "Beirut", Country = "Lebanon",
            About = "Lorem Ipsum"
        });
    await SendAsync(new DeleteTourListCommand
    {
        Id = listId
    });
    var list = await FindAsync<TourList>(listId);
    list.Should().BeNull();
}

```

The ShouldDeleteTourList list should be *null* because listId property was used to delete the TourList object.

TourLists/Commands/UpdateTourListTests

Next, we create a C# file named `UpdateTourListTests.cs` in the `Commands` folder of the `TourLists` directory and add the following code:

```
using FluentAssertions;
// Removed for brevity. Go to this repo's Github.
using Xunit;
namespace Application.IntegrationTests.TourLists.Commands
{
    using static DataFixture;
    [Collection("DatabaseCollection")]
    public class UpdateTourListTests
    {
        public UpdateTourListTests()
        {
            ResetState().GetAwaiter().GetResult();
        }
        [Fact]
        public void ShouldRequiredValidTourListId()
        {
            var command = new UpdateTourListCommand
            {
                Id = 8, City = "Caracas",
                Country = "Venezuela", About = "Lorem Ipsum"
            };
            FluentActions.Invoking(() =>
                SendAsync(command)).Should()
                .Throw<NotFoundException>();
        }
        // another test method
    }
}
```

`ShouldRequiredValidTourListId` should throw a *not found* exception in the preceding code because it tried to update a non-existing `TourList` object.

For the next test method, `ShouldUpdateTourList`, run the following code:

```
[Fact]
public async Task ShouldUpdateTourList()
{
    var listId = await SendAsync(new
        CreateTourListCommand
    {
        City = "Al Ghanim", Country = "Qatar",
        About = "Lorem Ipsum"
    });
    var command = new UpdateTourListCommand
    {
        Id = listId, City = "Doha",
        Country = "Qatar", About = "Lorem Ipsum"
    };
    await SendAsync(command);
    var list = await FindAsync<TourList>(listId);
    list.City.Should().Be(command.City);
    list.Country.Should().Be(command.Country);
    list.About.Should().NotNull();
}
```

Conversely, the `ShouldUpdateTourList` test should not be *null* after calling the `FindAsync` method because a `TourList` object was created and updated.

Now, let's create a `TourPackages` folder in the root directory of the `Application.IntegrationTests` project. Then, create a folder called `Commands` inside the newly created `TourPackages` folder.

TourPackages/Commands/CreateTourPackageTests

Next, we create a C# file named `CreateTourPackageTests.cs` in the `Commands` folder of the `TourPackages` directory and add the following code:

```
using FluentAssertions;
// Removed for brevity. Go to this repo's Github.
using Xunit;
namespace Application.IntegrationTests.TourPackages.Commands
{
```

```
using static DataFixture;
[Collection("DatabaseCollection")]
public class CreateTourPackageTests
{
    public CreateTourPackageTests()
    {
        ResetState().GetAwaiter().GetResult();
    }
    [Fact]
    public void ShouldRequireMinimumFields()
    {
        var command = new CreateTourPackageCommand();
        FluentActions.Invoking(() =>
                                SendAsync(command)).Should()
                                .Throw<ValidationException>();
    }
    // next test method here
}
}
```

`ShouldRequireMinimumFields` should throw a validation exception in the preceding code because it tried to create a `TourPackage` object without the required properties.

For the next test method, `ShouldCreateTourPackage`, run the following code:

```
[Fact]
public async Task ShouldCreateTourPackage()
{
    var listId = await SendAsync(new
        CreateTourListCommand
        {
            City = "New York", Country = "USA",
            About = "Lorem Ipsum"
        });
    var command = new CreateTourPackageCommand
    {
        ListId = listId,
```

```

        // Removed for brevity. Go to this repo's Github.
        WhatToExpect = "Lorem Ipsum"
    };
    var packageId = await SendAsync(command);
    var package = await
        FindAsync<TourPackage>(packageId);
    package.Should().NotNull();
    package.ListId.Should().Be(command.ListId);
    package.Name.Should().Be(command.Name);
    // Removed for brevity. Go to this repo's Github.
    package.WhatToExpect
        .Should().Be(command.WhatToExpect);
}

```

Then, `ShouldCreateTourPackage` should create a `TourPackage` object and find it because all required properties are filled.

TourPackages/Commands/DeleteTourPackageTests

Next, we create a C# file named `DeleteTourPackageTests.cs` in the `Commands` folder of the `TourPackages` directory and add the following code:

```

using System.Threading.Tasks;
// Removed for brevity. Go to this repo's Github.
using Xunit;
namespace Application.IntegrationTests.TourPackages.Commands
{
    using static DataFixture;
    [Collection("DatabaseCollection")]
    public class DeleteTourPackageTests
    {
        public DeleteTourPackageTests()
        {
            ResetState().GetAwaiter().GetResult();
        }
        [Fact]
        public void ShouldRequireValidTourPackageId()
        {

```

```
        var command = new DeleteTourPackageCommand
        {
            Id = 69
        };
        FluentActions.Invoking(() =>
            SendAsync(command)).Should()
            .Throw<NotFoundException>();
    }
    // second test method here
}
}
```

`ShouldRequireValidTourPackageId` should throw a *not found* exception in the preceding code because it tried to delete a non-existing `TourPackage` object.

For the second test method, `ShouldDeleteTourPackage`, run the following code:

```
[Fact]
public async Task ShouldDeleteTourPackage()
{
    var listId = await SendAsync(new
        CreateTourListCommand
        {
            City = "Tashkent", Country = "Uzbekistan",
            About = "Lorem Ipsum"
        });
    var packageId = await SendAsync(new
        CreateTourPackageCommand
        {
            ListId = listId, Name = "Silk Road Adventures",
            // Removed for brevity. Go to this repo's Github.
            WhatToExpect = "Lorem Ipsum"
        });
    await SendAsync(new DeleteTourPackageCommand
    {
        Id = packageId
    });
    var list = await FindAsync<TourPackage>(listId);
```

```
list.Should().BeNull();
}
```

On the flip side, `ShouldDeleteTourPackage` should create a `TourPackage` object, delete it, and receive *null* after calling the `FindAsync` method.

TourPackages/Commands/UpdateTourPackageDetailTests

Next, we create a C# file named `UpdateTourPackageDetailTests.cs` in the `Commands` folder of the `TourPackages` directory and add the following code:

```
using FluentAssertions;
using Travel.Application.Common.Exceptions;
using Xunit;
// Removed for brevity. Go to this repo's Github.
namespace Application.IntegrationTests.TourPackages.Commands
{
    using static DataFixture;
    [Collection("DatabaseCollection")]
    public class UpdateTourPackageDetailTests
    {
        public UpdateTourPackageDetailTests()
        {
            ResetState().GetAwaiter().GetResult();
        }
        [Fact]
        public void ShouldRequireValidTourPackageId()
        {
            var command = new UpdateTourPackageCommand
            {
                Id = 88,
                Name = "Free Walking Tour"
            };
            FluentActions.Invoking(() => SendAsync(command))
                .Should()
                .Throw<NotFoundException>();
        }
        // another test method here
    }
}
```

```
    }  
}
```

`ShouldRequireValidTourPackageId` should throw a *not found* exception in the preceding code because it tried to update a non-existing `TourPackage` object.

For the next test method, `ShouldUpdateTourPackage`, run the following code:

```
[Fact]  
public async Task ShouldUpdateTourPackage()  
{  
    var listId = await SendAsync(new  
        CreateTourListCommand  
    {  
        City = "Zagreb", Country = "Croatia",  
        About = "Lorem Ipsum"  
    });  
    var packageId = await SendAsync(new  
        CreateTourPackageCommand  
    {  
        ListId = listId,  
        Name = "Free Walking Tour Zagreb",  
        // Removed for brevity. Go to this repo's Github.  
        WhatToExpect = "Lorem Ipsum"  
    });  
    var command = new UpdateTourPackageDetailCommand  
    {  
        Id = packageId,  
        ListId = listId,  
        // Removed for brevity. Go to this repo's Github.  
        WhatToExpect = "Lorem Ipsum"  
    };  
    await SendAsync(command);  
    var item = await  
        FindAsync<TourPackage>(packageId);  
    item.ListId.Should().Be(command.ListId);  
    // Removed for brevity. Go to this repo's Github.  
}
```

```

        .Be (command.WhatToExpect) ;
    }

```

In contrast, the `ShouldUpdateTourPackage` method should create a `TourPackage` object, update it, and find it because all required properties and validations have been met.

TourPackages/Commands/UpdateTourPackageTests

Next, we create a C# file named `UpdateTourPackageTests.cs` in the `Commands` folder of the `TourPackages` directory and add the following code:

```

using Travel.Application.Common.Exceptions;
// Removed for brevity. Go to this repo's Github.
using Xunit;
namespace Application.IntegrationTests.TourPackages.Commands
{
    using static DataFixture;
    [Collection("DatabaseCollection")]
    public class UpdateTourPackageTests
    {
        public UpdateTourPackageTests()
        {
            ResetState().GetAwaiter().GetResult();
        }
        [Fact]
        public void ShouldRequireValidTourPackageId()
        {
            var command = new UpdateTourPackageCommand
            {
                Id = 4, Name = "Free Walking Tour"
            };
            FluentActions.Invoking(() =>
                SendAsync(command)).Should()
                .Throw<NotFoundException>();
        }
        // two more test methods here..
    }
}

```

In the preceding code, the `ShouldRequireValidTourPackageId` test method should throw a *not found* exception because it tried to update a non-existent `TourPackage` object.

For the next test method, `ShouldUpdateTourPackage`, copy the following code:

```
[Fact]
public async Task ShouldUpdateTourPackage()
{
    var listId = await SendAsync(new
        CreateTourListCommand
        {
            City = "Rabat", Country = "Morocco",
            About = "Lorem Ipsum"
        });
    var packageId = await SendAsync(new
        CreateTourPackageCommand
        {
            ListId = listId,
            Name = "Free Walking Tour Rabat",
            // Removed for brevity. Go to this repo's Github.
            WhatToExpect = "Lorem Ipsum"
        });
    var command = new UpdateTourPackageCommand
    {
        Id = packageId,
        Name = "Night Free Walking Tour Rabat."
    };
    await SendAsync(command);
    var item = await
        FindAsync<TourPackage>(packageId);
    item.Name.Should().Be(command.Name);
    item.WhatToExpect.Should().NotNull();
}
```

ShouldUpdateTourPackage should create a TourList object with a TourPackage object as a property, update TourPackage, and see it after invoking the FindAsync method because all requirements have been met.

For the last test method, ShouldRequireUniqueName, copy the following code:

```
[Fact]
public async Task ShouldRequireUniqueName()
{
    var listId = await SendAsync(new
        CreateTourListCommand
    {
        City = "Bogota", Country = "Colombia",
        About = "Lorem Ipsum"
    });
    await SendAsync(new CreateTourPackageCommand
    {
        ListId = listId, Name = "Bike Tour in Bogota"
    });
    await SendAsync(new CreateTourPackageCommand
    {
        ListId = listId, Name = "Salt Cathedral Tour"
    });
    var command = new UpdateTourPackageCommand
    {
        Id = listId, Name = "Salt Cathedral Tour"
    };
    FluentActions.Invoking(() =>
        SendAsync(command)).Should()
        .Throw<ValidationException>()
        .Where(ex => ex.Errors.ContainsKey("Name"))
        .And.Errors["Name"].Should()
        .Contain("The specified name already
            exists.");
}
```

The `ShouldRequireUniqueName` method should throw a validation exception that contains a `Name` key because it tried to use a name that already exists in another `TourPackage` object. We've written all the necessary tests that we need in our application. Now, go to the root directory of our solution, rebuild the application, and run the following `dotnet` command:

```
dotnet test
```

The preceding `dotnet test` command will run the tests of the `Application.IntegrationTests` and `Application.UnitTests` projects.

The results of running all the tests are shown here:

```
Passed! - Failed:    0, Passed:    18, Skipped:    0, Total:
18, Duration:    2 s
```

There are no failing tests. It means 0 tests failed, 18 tests passed, and there were no skipped tests. Plus, all of them ran for a total of 2 seconds, which is fast. They might run for 3 to 5 seconds, depending on your computer's specifications or open apps on your computer.

You can also run all the tests using the **Test Explorer** in your IDE.

Here is the result of running the tests on Visual Studio 2019's **Test Explorer**:

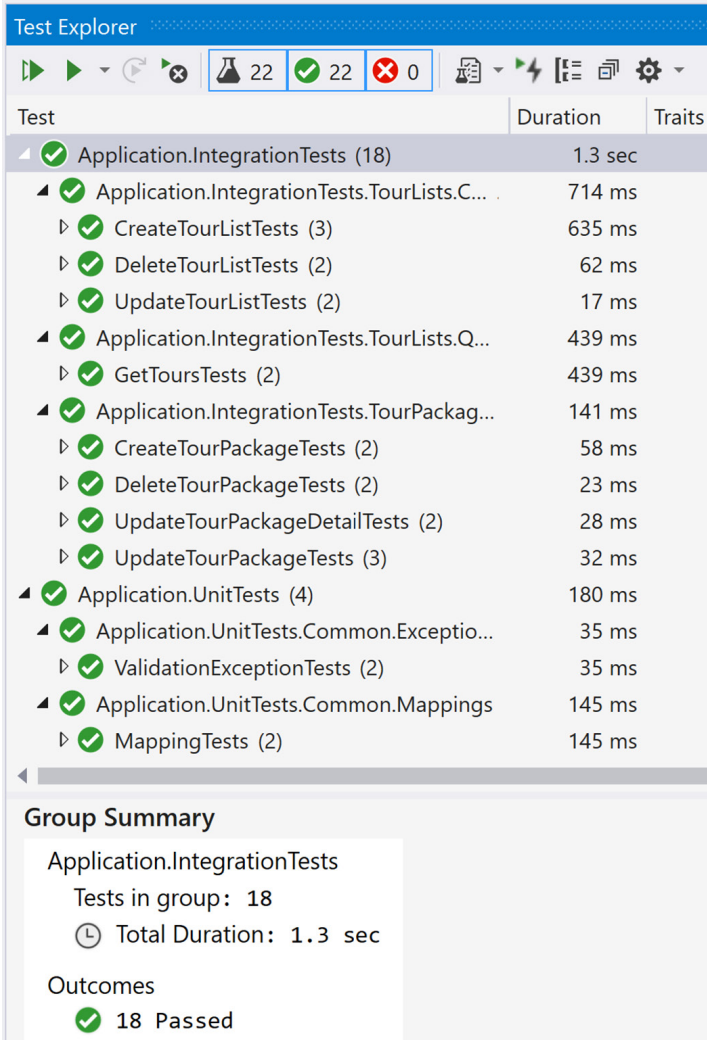


Figure 18.3 – Passed tests on Visual Studio 2019's Test Explorer

The preceding screenshot shows that all the tests are passing on Visual Studio 2019.

Here is the result of running the tests in Visual Studio for Mac's **Test Explorer**:

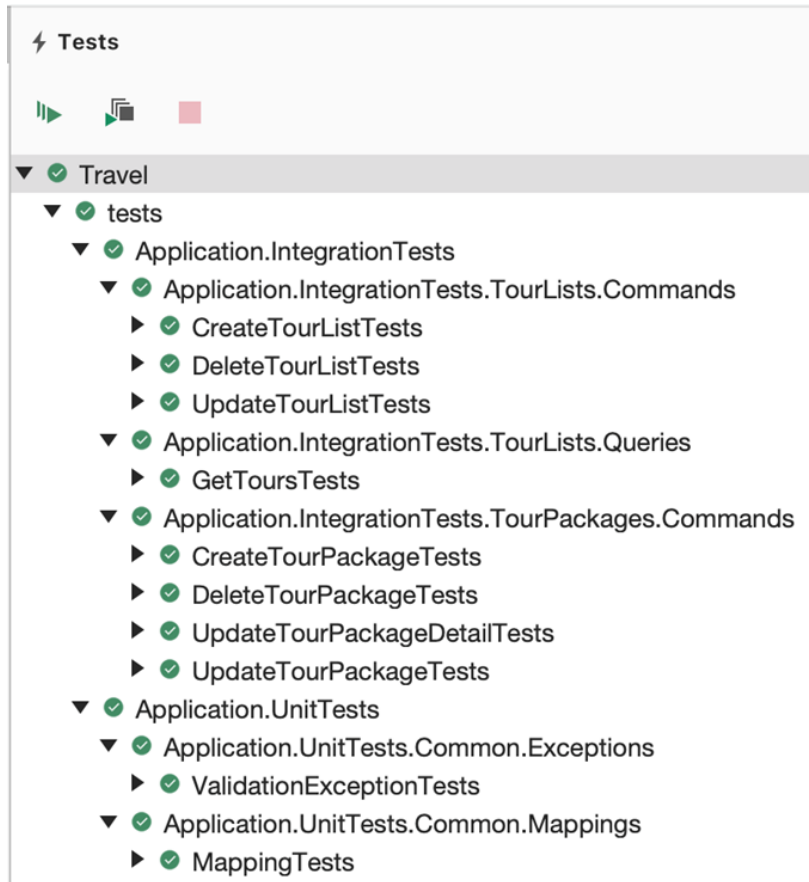


Figure 18.4 – Passed tests on Visual Studio for Mac's Test Explorer

The following screenshot shows the result of running the tests in JetBrains Rider's **Test Explorer**:

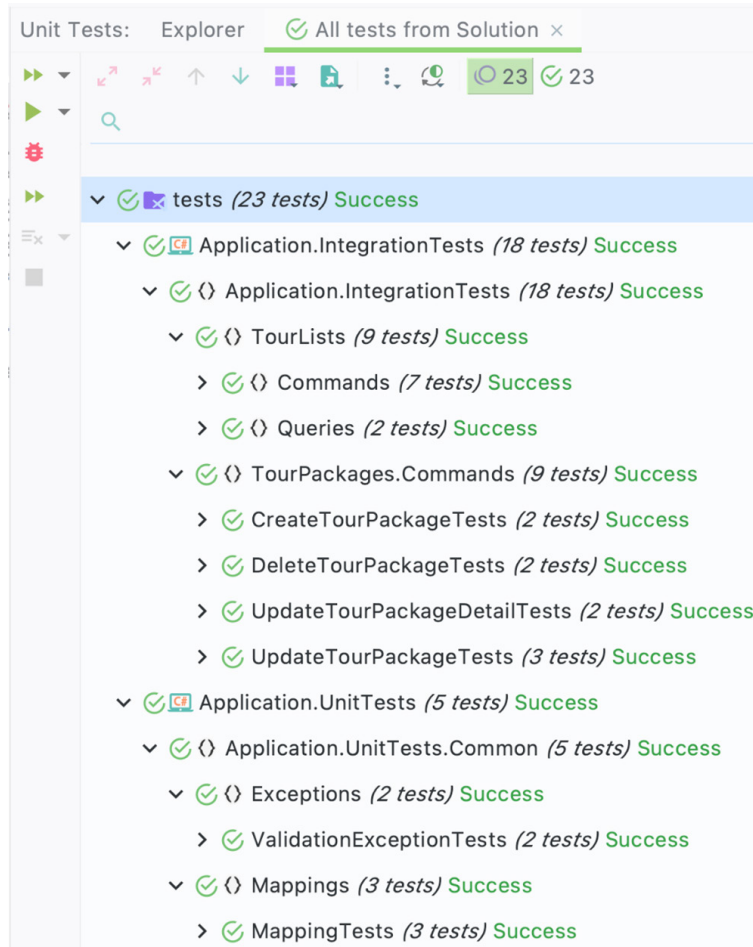


Figure 18.5 – Passed tests on JetBrains Rider's Test Explorer

And that's it! We have written automated tests using xUnit and have written these in a pragmatic way, but packed with best practices and using the latest packages/libraries in our tests.

However, I still encourage you to read more about writing tests using xUnit to solidify what you have learned here. Here are some references:

- **Testing in .NET:** <https://docs.microsoft.com/en-us/dotnet/core/testing/>
- **Using xUnit to test your C# code:** <https://auth0.com/blog/xunit-to-test-csharp-code/>
- **xUnit.net:** <https://xunit.net/>

You will see that the assertions in the tests from the preceding links are not `FluentAssertions`. To test what you have learned, you need to do some exercises.

Activity

Convert all the xUnit assertions that you see in the test samples of the link to `FluentAssertions`.

This was indeed an excellent chapter for busy developers such as you. Now, let's sum up what we have tackled.

Summary

You have learned what automated testing is and the value that it brings to software development. You've learned how to install a MS SQL Server database in a Docker container for Linux or macOS users. You've learned what xUnit is and why it is suited to building modern .NET apps, and how to use xUnit with `FluentAssertions`. You've also learned what are the pros and cons of unit and integration testing.

In the next chapter, we will do ASP.NET Core and Vue.js app deployments in containers and in the Azure Cloud.